Cognome.....

Nome.....

Matricola.....

Algoritmi e Strutture Dati

Prova scritta del 17 gennaio 2025

TEMPO DISPONIBILE: 2 ore

Le risposte agli esercizi 1 e 2 devono essere scritte negli appositi riquadri su questo foglio (risposte scritte su altri fogli non saranno considerate). La soluzione dell'esercizio 3 va scritta su uno dei fogli di protocollo forniti. Le brutte copie NON devono essere consegnate. Ricordatevi di scrivere cognome e nome su TUTTI i fogli (inclusi quelli di brutta).

1. Considerate la sequenza di numeri `18 43 52 27 11 14 19 23`.

(a) Disegnate l'albero AVL che si ottiene inserendo i numeri della sequenza, uno dopo l'altro, nell'ordine indicato, a partire da un albero inizialmente vuoto, ribilanciando quando necessario, in modo da ottenere un albero AVL dopo ogni inserimento.	(b) Disegnate un albero 2-3 che contenga i numeri della sequenza e abbia un numero di nodi minimo.

2. Considerate la seguente sequenza di numeri, memorizzata in un array: `14 19 23 11 45 18 43 52`.

(a) Supponete di ordinare la sequenza *in modo crescente* mediante l'algoritmo `quickSort`, scegliendo come perno il primo elemento ed utilizzando, per effettuare la partizione, la strategia presentata a lezione.
Indicate il contenuto dell'array dopo avere effettuato la partizione, prima delle chiamate ricorsive di `quickSort`.

(b) Supponete di ordinare la sequenza *in modo decrescente* mediante l'algoritmo `quickSort`, scegliendo come perno il primo elemento ed utilizzando, per effettuare la partizione, la strategia presentata a lezione.
Indicate il contenuto dell'array dopo avere effettuato la partizione, prima delle chiamate ricorsive di `quickSort`.

(c) Supponete di ordinare la sequenza *in modo non decrescente* mediante l'algoritmo `heapSort`.
Indicate il contenuto dell'array *immediatamente* dopo lo scambio che colloca `45` nella sua posizione definitiva.

(d) Supponete di dividere l'array in due parti formate rispettivamente dagli elementi nelle prime 5 e nelle ultime 3 posizioni e di applicare a esse l'algoritmo di `merge`, *senza averle ordinate*.
Indicate la sequenza che si ottiene.

3. Dato un grafo $G = (V, E)$ con una funzione peso $\omega : E \to \mathbb{N}$, per ogni coppia di vertici $x, y \in V$ chiamiamo *distanza* da x a y il costo del cammino minimo che inizia in x e termina in y. Si supponga che $V = \{v_1, v_2, \ldots, v_n\}$ e che sia data una matrice D di n righe e n colonne nella quale l'elemento di riga i e colonna j, $i, j = 1, \ldots, n$, è definito come segue:

$$d_{i,j} = \begin{cases} 0 & \text{se } i = j, \\ \omega(v_i, v_j) & \text{se } i \neq j \text{ e } (v_i, v_j) \in E, \\ \infty & \text{altrimenti.} \end{cases}$$

Risolvete i seguenti punti ***nell'ordine indicato.***

(a) Si deve costruire un algoritmo che, *ricevendo in ingresso la matrice D*, restituisca due array di nomi `maxDa` e `maxA` i cui indici rappresentano i vertici e per `i` $= 1, \ldots, n$:

- `MaxDa[i]` contiene l'indice j del vertice v_j più distante da v_i,
- `MaxA[i]` contiene l'indice j del vertice v_j dal quale v_i è più distante,

nel caso vi siano più vertici j che soddisfano la condizione richiesta se ne fornisce uno qualunque di essi. L'algoritmo durante l'esecuzione può modificare il contenuto della matrice D ricevuta in ingresso.

Per la soluzione ispiratevi a uno degli algoritmi presentati a lezione: indicate quale e spiegate sinteticamente le modifiche da apportare all'algoritmo.

(b) Scrivete lo pseudocodice dell'algoritmo per risolvere il problema al punto (a).

(c) Fornite una stima del tempo di calcolo utilizzato dall'algoritmo che avete scritto in funzione di n, *giustificando adeguatamente la risposta.*

(d) Si deve produrre un elenco che indichi, per ogni coppia (i, j), la distanza dal vertice v_i al vertice v_j. L'elenco deve essere in ordine *non decrescente* rispetto alle distanze. Indicate come si può modificare ed estendere a questo scopo l'algoritmo ottenuto ai punti precedenti, eventualmente introducendo strutture dati aggiuntive. Potete richiamare un algoritmo di ordinamento noto (indicate quale, senza descriverne la strategia).

(e) Fornite una stima del tempo di calcolo della parte descritta al punto (d), in funzione del numero n di vertici.

Note

- Nella scrittura dello pseudocodice potete utilizzare il valore ∞.
- Le risposte devono essere adeguatamente giustificate.
- Non cambiate i nomi stabiliti nel testo dell'esercizio, in particolare D, `MaxDa` e `MaxA`, gli interi da 1 a n per indicare i vertici.
- Prestate attenzione a righe e colonne nelle matrici!

Esempio

Matrice D in ingresso:

$$\begin{bmatrix} 0 & 5 & \infty & \infty \\ \infty & 0 & 7 & 4 \\ \infty & 1 & 0 & \infty \\ 2 & \infty & 2 & 0 \end{bmatrix}$$

Distanze tra coppie di vertici:

$$\begin{bmatrix} 0 & 5 & 11 & 9 \\ 6 & 0 & 6 & 4 \\ 7 & 1 & 0 & 5 \\ 2 & 3 & 2 & 0 \end{bmatrix}$$

Array in uscita:

i	`MaxDa`	`MaxA`
1	3	3
2	1	1
3	1	1
4	2	1

Elenco ordinato (punto (d))

da	*a*	*distanza*
3	2	1
4	1	2
4	3	2
4	2	3
2	4	4
1	2	5
3	4	5
2	1	6
2	3	6
3	1	7
1	4	9
1	3	11

Cognome.....

Nome.....

Matricola.....

Algoritmi e Strutture Dati

Prova scritta del 17 gennaio 2025

TEMPO DISPONIBILE: 2 ore

Le risposte agli esercizi 1 e 2 devono essere scritte negli appositi riquadri su questo foglio (risposte scritte su altri fogli non saranno considerate). La soluzione dell'esercizio 3 va scritta su uno dei fogli di protocollo forniti. Le brutte copie NON devono essere consegnate. Ricordatevi di scrivere cognome e nome su TUTTI i fogli (inclusi quelli di brutta).

1. Considerate la sequenza di numeri `19 44 53 28 12 15 20 24`.

(a) Disegnate l'albero AVL che si ottiene inserendo i numeri della sequenza, uno dopo l'altro, nell'ordine indicato, a partire da un albero inizialmente vuoto, ribilanciando quando necessario, in modo da ottenere un albero AVL dopo ogni inserimento.	(b) Disegnate un albero 2-3 che contenga i numeri della sequenza e abbia un numero di nodi minimo.

2. Considerate la seguente sequenza di numeri, memorizzata in un array: `15 20 24 12 46 19 44 53`.

(a) Supponete di ordinare la sequenza *in modo crescente* mediante l'algoritmo `quickSort`, scegliendo come perno il primo elemento ed utilizzando, per effettuare la partizione, la strategia presentata a lezione.
Indicate il contenuto dell'array dopo avere effettuato la partizione, prima delle chiamate ricorsive di `quickSort`.

(b) Supponete di ordinare la sequenza *in modo decrescente* mediante l'algoritmo `quickSort`, scegliendo come perno il primo elemento ed utilizzando, per effettuare la partizione, la strategia presentata a lezione.
Indicate il contenuto dell'array dopo avere effettuato la partizione, prima delle chiamate ricorsive di `quickSort`.

(c) Supponete di ordinare la sequenza *in modo non decrescente* mediante l'algoritmo `heapSort`.
Indicate il contenuto dell'array *immediatamente* dopo lo scambio che colloca `46` nella sua posizione definitiva.

(d) Supponete di dividere l'array in due parti formate rispettivamente dagli elementi nelle prime 5 e nelle ultime 3 posizioni e di applicare a esse l'algoritmo di `merge`, *senza averle ordinate*.
Indicate la sequenza che si ottiene.

3. Dato un grafo $G = (V, E)$ con una funzione peso $\omega : E \to \mathbb{N}$, per ogni coppia di vertici $x, y \in V$ chiamiamo *distanza* da x a y il costo del cammino minimo che inizia in x e termina in y. Si supponga che $V = \{v_1, v_2, \ldots, v_n\}$ e che sia data una matrice D di n righe e n colonne nella quale l'elemento di riga i e colonna j, $i, j = 1, \ldots, n$, è definito come segue:

$$d_{i,j} = \begin{cases} 0 & \text{se } i = j, \\ \omega(v_i, v_j) & \text{se } i \neq j \text{ e } (v_i, v_j) \in E, \\ \infty & \text{altrimenti.} \end{cases}$$

Risolvete i seguenti punti ***nell'ordine indicato.***

(a) Si deve costruire un algoritmo che, *ricevendo in ingresso la matrice D*, restituisca due array di nomi `maxDa` e `maxA` i cui indici rappresentano i vertici e per `i` $= 1, \ldots, n$:

- `MaxDa[i]` contiene l'indice j del vertice v_j più distante da v_i,
- `MaxA[i]` contiene l'indice j del vertice v_j dal quale v_i è più distante,

nel caso vi siano più vertici j che soddisfano la condizione richiesta se ne fornisce uno qualunque di essi. L'algoritmo durante l'esecuzione può modificare il contenuto della matrice D ricevuta in ingresso.

Per la soluzione ispiratevi a uno degli algoritmi presentati a lezione: indicate quale e spiegate sinteticamente le modifiche da apportare all'algoritmo.

(b) Scrivete lo pseudocodice dell'algoritmo per risolvere il problema al punto (a).

(c) Fornite una stima del tempo di calcolo utilizzato dall'algoritmo che avete scritto in funzione di n, *giustificando adeguatamente la risposta.*

(d) Si deve produrre un elenco che indichi, per ogni coppia (i, j), la distanza dal vertice v_i al vertice v_j. L'elenco deve essere in ordine *non decrescente* rispetto alle distanze. Indicate come si può modificare ed estendere a questo scopo l'algoritmo ottenuto ai punti precedenti, eventualmente introducendo strutture dati aggiuntive. Potete richiamare un algoritmo di ordinamento noto (indicate quale, senza descriverne la strategia).

(e) Fornite una stima del tempo di calcolo della parte descritta al punto (d), in funzione del numero n di vertici.

Note

- Nella scrittura dello pseudocodice potete utilizzare il valore ∞.
- Le risposte devono essere adeguatamente giustificate.
- Non cambiate i nomi stabiliti nel testo dell'esercizio, in particolare D, `MaxDa` e `MaxA`, gli interi da 1 a n per indicare i vertici.
- Prestate attenzione a righe e colonne nelle matrici!

Esempio

Matrice D in ingresso:

$$\begin{bmatrix} 0 & 5 & \infty & \infty \\ \infty & 0 & 7 & 4 \\ \infty & 1 & 0 & \infty \\ 2 & \infty & 2 & 0 \end{bmatrix}$$

Distanze tra coppie di vertici:

$$\begin{bmatrix} 0 & 5 & 11 & 9 \\ 6 & 0 & 6 & 4 \\ 7 & 1 & 0 & 5 \\ 2 & 3 & 2 & 0 \end{bmatrix}$$

Array in uscita:

i	`MaxDa`	`MaxA`
1	3	3
2	1	1
3	1	1
4	2	1

Elenco ordinato (punto (d))

da	*a*	*distanza*
3	2	1
4	1	2
4	3	2
4	2	3
2	4	4
1	2	5
3	4	5
2	1	6
2	3	6
3	1	7
1	4	9
1	3	11

Cognome.....

Nome.....

Matricola.....

Algoritmi e Strutture Dati

Prova scritta del 17 gennaio 2025

TEMPO DISPONIBILE: 2 ore

Le risposte agli esercizi 1 e 2 devono essere scritte negli appositi riquadri su questo foglio (risposte scritte su altri fogli non saranno considerate). La soluzione dell'esercizio 3 va scritta su uno dei fogli di protocollo forniti. Le brutte copie NON devono essere consegnate. Ricordatevi di scrivere cognome e nome su TUTTI i fogli (inclusi quelli di brutta).

1. Considerate la sequenza di numeri `20 45 54 29 13 16 21 25`.

(a) Disegnate l'albero AVL che si ottiene inserendo i numeri della sequenza, uno dopo l'altro, nell'ordine indicato, a partire da un albero inizialmente vuoto, ribilanciando quando necessario, in modo da ottenere un albero AVL dopo ogni inserimento.	(b) Disegnate un albero 2-3 che contenga i numeri della sequenza e abbia un numero di nodi minimo.

2. Considerate la seguente sequenza di numeri, memorizzata in un array: `16 21 25 13 47 20 45 54`.

(a) Supponete di ordinare la sequenza *in modo crescente* mediante l'algoritmo `quickSort`, scegliendo come perno il primo elemento ed utilizzando, per effettuare la partizione, la strategia presentata a lezione.
Indicate il contenuto dell'array dopo avere effettuato la partizione, prima delle chiamate ricorsive di `quickSort`.

(b) Supponete di ordinare la sequenza *in modo decrescente* mediante l'algoritmo `quickSort`, scegliendo come perno il primo elemento ed utilizzando, per effettuare la partizione, la strategia presentata a lezione.
Indicate il contenuto dell'array dopo avere effettuato la partizione, prima delle chiamate ricorsive di `quickSort`.

(c) Supponete di ordinare la sequenza *in modo non decrescente* mediante l'algoritmo `heapSort`.
Indicate il contenuto dell'array *immediatamente* dopo lo scambio che colloca `47` nella sua posizione definitiva.

(d) Supponete di dividere l'array in due parti formate rispettivamente dagli elementi nelle prime 5 e nelle ultime 3 posizioni e di applicare a esse l'algoritmo di `merge`, *senza averle ordinate*.
Indicate la sequenza che si ottiene.

3. Dato un grafo $G = (V, E)$ con una funzione peso $\omega : E \to \mathbb{N}$, per ogni coppia di vertici $x, y \in V$ chiamiamo *distanza* da x a y il costo del cammino minimo che inizia in x e termina in y. Si supponga che $V = \{v_1, v_2, \ldots, v_n\}$ e che sia data una matrice D di n righe e n colonne nella quale l'elemento di riga i e colonna j, $i, j = 1, \ldots, n$, è definito come segue:

$$d_{i,j} = \begin{cases} 0 & \text{se } i = j, \\ \omega(v_i, v_j) & \text{se } i \neq j \text{ e } (v_i, v_j) \in E, \\ \infty & \text{altrimenti.} \end{cases}$$

Risolvete i seguenti punti ***nell'ordine indicato.***

(a) Si deve costruire un algoritmo che, *ricevendo in ingresso la matrice D*, restituisca due array di nomi `maxDa` e `maxA` i cui indici rappresentano i vertici e per `i` $= 1, \ldots, n$:

- `MaxDa[i]` contiene l'indice j del vertice v_j più distante da v_i,
- `MaxA[i]` contiene l'indice j del vertice v_j dal quale v_i è più distante,

nel caso vi siano più vertici j che soddisfano la condizione richiesta se ne fornisce uno qualunque di essi. L'algoritmo durante l'esecuzione può modificare il contenuto della matrice D ricevuta in ingresso.

Per la soluzione ispiratevi a uno degli algoritmi presentati a lezione: indicate quale e spiegate sinteticamente le modifiche da apportare all'algoritmo.

(b) Scrivete lo pseudocodice dell'algoritmo per risolvere il problema al punto (a).

(c) Fornite una stima del tempo di calcolo utilizzato dall'algoritmo che avete scritto in funzione di n, *giustificando adeguatamente la risposta.*

(d) Si deve produrre un elenco che indichi, per ogni coppia (i, j), la distanza dal vertice v_i al vertice v_j. L'elenco deve essere in ordine *non decrescente* rispetto alle distanze. Indicate come si può modificare ed estendere a questo scopo l'algoritmo ottenuto ai punti precedenti, eventualmente introducendo strutture dati aggiuntive. Potete richiamare un algoritmo di ordinamento noto (indicate quale, senza descriverne la strategia).

(e) Fornite una stima del tempo di calcolo della parte descritta al punto (d), in funzione del numero n di vertici.

Note

- Nella scrittura dello pseudocodice potete utilizzare il valore ∞.
- Le risposte devono essere adeguatamente giustificate.
- Non cambiate i nomi stabiliti nel testo dell'esercizio, in particolare D, `MaxDa` e `MaxA`, gli interi da 1 a n per indicare i vertici.
- Prestate attenzione a righe e colonne nelle matrici!

Esempio

Matrice D in ingresso:

$$\begin{bmatrix} 0 & 5 & \infty & \infty \\ \infty & 0 & 7 & 4 \\ \infty & 1 & 0 & \infty \\ 2 & \infty & 2 & 0 \end{bmatrix}$$

Distanze tra coppie di vertici:

$$\begin{bmatrix} 0 & 5 & 11 & 9 \\ 6 & 0 & 6 & 4 \\ 7 & 1 & 0 & 5 \\ 2 & 3 & 2 & 0 \end{bmatrix}$$

Array in uscita:

i	`MaxDa`	`MaxA`
1	3	3
2	1	1
3	1	1
4	2	1

Elenco ordinato (punto (d))

da	*a*	*distanza*
3	2	1
4	1	2
4	3	2
4	2	3
2	4	4
1	2	5
3	4	5
2	1	6
2	3	6
3	1	7
1	4	9
1	3	11

Cognome.....

Nome.....

Matricola.....

Algoritmi e Strutture Dati

Prova scritta del 17 gennaio 2025

TEMPO DISPONIBILE: 2 ore

Le risposte agli esercizi 1 e 2 devono essere scritte negli appositi riquadri su questo foglio (risposte scritte su altri fogli non saranno considerate). La soluzione dell'esercizio 3 va scritta su uno dei fogli di protocollo forniti. Le brutte copie NON devono essere consegnate. Ricordatevi di scrivere cognome e nome su TUTTI i fogli (inclusi quelli di brutta).

1. Considerate la sequenza di numeri `21 46 55 30 14 17 22 26`.

(a) Disegnate l'albero AVL che si ottiene inserendo i numeri della sequenza, uno dopo l'altro, nell'ordine indicato, a partire da un albero inizialmente vuoto, ribilanciando quando necessario, in modo da ottenere un albero AVL dopo ogni inserimento.	(b) Disegnate un albero 2-3 che contenga i numeri della sequenza e abbia un numero di nodi minimo.

2. Considerate la seguente sequenza di numeri, memorizzata in un array: `17 22 26 14 48 21 46 55`.

(a) Supponete di ordinare la sequenza *in modo crescente* mediante l'algoritmo `quickSort`, scegliendo come perno il primo elemento ed utilizzando, per effettuare la partizione, la strategia presentata a lezione.
Indicate il contenuto dell'array dopo avere effettuato la partizione, prima delle chiamate ricorsive di `quickSort`.

(b) Supponete di ordinare la sequenza *in modo decrescente* mediante l'algoritmo `quickSort`, scegliendo come perno il primo elemento ed utilizzando, per effettuare la partizione, la strategia presentata a lezione.
Indicate il contenuto dell'array dopo avere effettuato la partizione, prima delle chiamate ricorsive di `quickSort`.

(c) Supponete di ordinare la sequenza *in modo non decrescente* mediante l'algoritmo `heapSort`.
Indicate il contenuto dell'array *immediatamente* dopo lo scambio che colloca `48` nella sua posizione definitiva.

(d) Supponete di dividere l'array in due parti formate rispettivamente dagli elementi nelle prime 5 e nelle ultime 3 posizioni e di applicare a esse l'algoritmo di `merge`, *senza averle ordinate*.
Indicate la sequenza che si ottiene.

3. Dato un grafo $G = (V, E)$ con una funzione peso $\omega : E \to \mathbb{N}$, per ogni coppia di vertici $x, y \in V$ chiamiamo *distanza* da x a y il costo del cammino minimo che inizia in x e termina in y. Si supponga che $V = \{v_1, v_2, \ldots, v_n\}$ e che sia data una matrice D di n righe e n colonne nella quale l'elemento di riga i e colonna j, $i, j = 1, \ldots, n$, è definito come segue:

$$d_{i,j} = \begin{cases} 0 & \text{se } i = j, \\ \omega(v_i, v_j) & \text{se } i \neq j \text{ e } (v_i, v_j) \in E, \\ \infty & \text{altrimenti.} \end{cases}$$

Risolvete i seguenti punti ***nell'ordine indicato.***

(a) Si deve costruire un algoritmo che, *ricevendo in ingresso la matrice D*, restituisca due array di nomi `maxDa` e `maxA` i cui indici rappresentano i vertici e per `i` $= 1, \ldots, n$:

- `MaxDa[i]` contiene l'indice j del vertice v_j più distante da v_i,
- `MaxA[i]` contiene l'indice j del vertice v_j dal quale v_i è più distante,

nel caso vi siano più vertici j che soddisfano la condizione richiesta se ne fornisce uno qualunque di essi. L'algoritmo durante l'esecuzione può modificare il contenuto della matrice D ricevuta in ingresso.

Per la soluzione ispiratevi a uno degli algoritmi presentati a lezione: indicate quale e spiegate sinteticamente le modifiche da apportare all'algoritmo.

(b) Scrivete lo pseudocodice dell'algoritmo per risolvere il problema al punto (a).

(c) Fornite una stima del tempo di calcolo utilizzato dall'algoritmo che avete scritto in funzione di n, *giustificando adeguatamente la risposta.*

(d) Si deve produrre un elenco che indichi, per ogni coppia (i, j), la distanza dal vertice v_i al vertice v_j. L'elenco deve essere in ordine *non decrescente* rispetto alle distanze. Indicate come si può modificare ed estendere a questo scopo l'algoritmo ottenuto ai punti precedenti, eventualmente introducendo strutture dati aggiuntive. Potete richiamare un algoritmo di ordinamento noto (indicate quale, senza descriverne la strategia).

(e) Fornite una stima del tempo di calcolo della parte descritta al punto (d), in funzione del numero n di vertici.

Note

- Nella scrittura dello pseudocodice potete utilizzare il valore ∞.
- Le risposte devono essere adeguatamente giustificate.
- Non cambiate i nomi stabiliti nel testo dell'esercizio, in particolare D, `MaxDa` e `MaxA`, gli interi da 1 a n per indicare i vertici.
- Prestate attenzione a righe e colonne nelle matrici!

Esempio

Matrice D in ingresso:

$$\begin{bmatrix} 0 & 5 & \infty & \infty \\ \infty & 0 & 7 & 4 \\ \infty & 1 & 0 & \infty \\ 2 & \infty & 2 & 0 \end{bmatrix}$$

Distanze tra coppie di vertici:

$$\begin{bmatrix} 0 & 5 & 11 & 9 \\ 6 & 0 & 6 & 4 \\ 7 & 1 & 0 & 5 \\ 2 & 3 & 2 & 0 \end{bmatrix}$$

Array in uscita:

i	`MaxDa`	`MaxA`
1	3	3
2	1	1
3	1	1
4	2	1

Elenco ordinato (punto (d))

da	*a*	*distanza*
3	2	1
4	1	2
4	3	2
4	2	3
2	4	4
1	2	5
3	4	5
2	1	6
2	3	6
3	1	7
1	4	9
1	3	11